



北京邮电大学

Beijing University of Posts and Telecommunications



北京邮电大学

Beijing University of Posts and Telecommunications

项目式课程阶段2

--从部件到整机：计算机系统控制器与处理器设计实践

计算机学院（国家示范性软件学院）实验中心

杨秦 youngq@bupt.edu.cn

时间：2026年7月

目录

Contents

- 1 课程概述
- 2 课程实施与考核要求
- 3 课程设计任务与要求
- 4 设计平台选择与特性对比
- 5 设计思路与技术要点
- 6 Verilog工程应用技巧与规范
- 7 附录：平台资源与参考资料



课程概述

课程性质： 本课程是计算机组成理论的实践延伸，要求学生利用HDL与EDA工具，从基础CPU控制器出发，逐步设计并实现可支持小型操作系统与应用软件运行的完整计算机系统。在AI大模型广泛应用的背景下，系统性能的瓶颈日益从算法层下移至硬件层，体现在计算效率、存储带宽与资源利用率等方面。具备从需求分析到硬件落地的**完整工程能力**，是AI时代难以被替代的核心竞争力。

核心目标： 以模型计算机系统及RISC-V等主流架构为参照，在资源与时序约束下，通过设计指令集、数据通路与I/O接口，经历需求分析、方案设计、迭代调试、系统验证的完整工程闭环，实现从单一部件到整机系统的认知跨越；在此基础上引入流水线技术，完成系统性能的量化优化，建立**软硬协同的系统级工程视角**。

课程重点： **团队分工与工程协作规范**；工程项目迭代开发—从基础CPU控制器到完整计算机系统的阶梯式推进，历经功能实现、时序收敛、资源优化、性能提升的多轮迭代；硬件约束下复杂工程问题的定位、分析与解决；弥合软硬件认知断层，培养**科研思维与工程实践能力**；善用开源实现与AI工具、具备工程审阅与系统整合能力，能清楚说明自己在系统中做出的判断与贡献。



计算机系统设计——AI时代的工程师底色

从“实现CPU”到“理解AI芯片”：GPU、TPU、NPU的本质，是针对特定计算模式深度优化的处理器架构。它们的数据通路设计、指令调度逻辑与存储层次结构，与本课程所实现的CPU在架构原理上一脉相承。当你完成一个可运行的CPU设计，你所建立的不只是对模型机的理解，而是**读懂任何处理器架构**的基础语言。

从“跑通指令”到“量化优化”：从AI时代的工程竞争力，不在于会调用多少API，而在于能否理解系统性能从何而来、受制于何处。时钟频率、流水线吞吐量、访存延迟——这些指标在本课程中都有直接对应的设计决策。学会在**资源与时序约束下进行量化权衡**，是从“会用工具”迈向能“优化系统”的关键一步。

从“单人编码”到“系统工程”：大模型的广泛应用正在重塑软件开发方式，但也在放大系统复杂度——更多的模块、更长的调用链、更难定位的性能瓶颈，也让“独立编写”与“审阅整合他人/AI生成代码”同样成为核心工程能力。在本课程中，你将经历模块分工、接口对齐、联调集成的完整团队工程流程。**理解硬件行为边界、具备跨层次的系统调试能力，是构建可靠复杂系统不可或缺的工程素养。**





课程设计教学安排

- 按照课程设计任务要求，选课同学按4人为一组**分组**，确定组长，建立小组内部沟通与讨论渠道，并在教学云平台完成小组信息登记。
- 各小组在充分讨论的基础上确定**设计题目**，完成相关资料的查阅与下载，完成所选**设计平台**的软件环境安装与配置，并在组内明确分工，制定初步实施计划。
- 课程设计过程中，各小组须全过程记录**调试日志**，包括但不限于总体设计思路、实施进度安排、关键问题与故障分析及解决方案、AI辅助工具与开源参考的使用情况（详见第13页声明表），同时反映小组成员的讨论与协作情况。
- 实验室在课程设计期间按照教学安排实行开放管理，课程设计负责教师定期组织集中答疑与指导。课程设置**中期检查与结题验收**答辩两个关键环节，对各小组的阶段成果和最终完成情况进行检查与评定。（注：**中期检查为过程性考核，不单独计分。**）
- 各小组须按照统一要求撰写并提交**课程设计报告**。



课程设计验收答辩

- 课程设计验收答辩在各小组完成系统设计与调试后进行，作为工程项目的最终验收环节。
- 各小组须在验收前完成基于VHDL或Verilog的工程设计工作，包括工程建立、编译、功能仿真及系统调试，确保设计功能正确、运行状态稳定。
- 验收过程中，小组成员须全部到场。验收开始前，各小组需提交并出示完整的调试日志以及小组成员分工与贡献度说明表，作为过程性考核的重要依据。调试日志须如实记录设计过程中AI辅助工具与开源参考的使用情况，包括使用环节、生成/参考内容与人工审阅整合过程（题目A、题目B标准不同，详见第13页）；**未按要求记录或未经审阅整合直接提交的AI生成代码/开源代码，将按学术诚信要求处理。**
- 验收答辩环节中，小组需向评审教师现场演示系统功能与运行状态，说明设计目标的实现情况及关键技术指标。答辩过程中，小组各成员需分别讲解本人负责的功能模块，阐述设计思路、实现方法及调试要点。
- 评审教师将通过现场提问的方式，重点考察学生对系统整体结构与关键技术细节的理解程度，以及能否独立解释本人负责模块的设计依据（无论该模块是自主实现还是审阅整合开源/AI成果），以此甄别工具辅助与工程判断的边界，全面评估工程实现与问题分析能力



课程设计报告要求

- ◆ 各小组须撰写并提交一份完整、规范的课程设计报告，作为课程设计成果的重要组成部分。
- ◆ 每个小组须同时提交以组号+班级命名的两份电子文件，具体包括：课程设计源代码文件（压缩包形式）以及课程设计报告电子版（非压缩文档）。
- ◆ 课程设计报告须采用学校统一规定的课程设计封面格式，报告正文应按章节结构展开，内容完整、条理清晰。课程设计报告内容分为报告主体和报告附页两部分：
 - ✓ **报告主体：**1.课题所使用的硬件/实验环境及系统总体设计说明。2.课题需求分析与设计目标。3.系统概要设计与总体结构。4.各功能模块的详细设计与实现方法。5.设计与调试过程中出现的主要问题、分析过程及解决方案（如借助AI工具或开源参考辅助分析、调试或实现，须说明具体使用环节及人工验证/审阅整合过程）。6.课程设计调试总结与效果分析。
 - ✓ **报告附页：**1.小组各成员个人心得与总结。2.小组完整调试日志记录（须如实记录AI辅助工具与开源参考使用的具体环节、生成/参考内容与人工审阅整合过程。3.AI辅助工具使用声明表(如有使用请附上简表) 4.小组成员分工与贡献度说明表



成绩构成与评分规则

1. 总评成绩 = (系统演示与答辩验收成绩70%+ 小组报告成绩30%) *调节因子 **注：个人最终得分将结合个人贡献度等调节因子进行加权调整，组内成员分数可能存在差异。**

2. 评分细则：

A：系统演示与答辩验收（70%） 重点考核学生在课程设计过程中的工程实现能力与技术掌握程度，主要评价内容包括：系统整体设计思路与完成度；功能实现的正确性与稳定性；附加功能实现情况及创新性；硬件资源利用效率；系统运行状态与调试效果；问题分析与解决能力；参考内容的工程审阅与系统整合质量（如涉及开源/AI参考）；系统演示表现及答辩情况。

B：课程设计报告（30%） 主要考核学生的技术文档撰写能力，重点评价课程设计报告的：内容完整性与规范性；技术描述的准确性；结构安排与逻辑性；图表、代码说明及附录材料的质量；AI辅助工具与开源参考使用记录的真实性、完整性与审阅说明质量。

3. **学术诚信与否决项【红线规则】**：课程设计严格遵守学术诚信与工程规范要求。若发现课程设计代码或报告中存在严重抄袭行为（包括但不限于**组间抄袭、网络抄袭、不当使用AI生成内容或未经审阅整合直接照搬开源代码等**），一经查实，将直接取消该组或相关个人的本课程成绩（记为0分），并按学校有关考试违纪与学术不端的相关规定处理。





任务总览:

- ✓ 本课程设计围绕两类核心题目展开，学生二选一，每类题目均设基础、进阶、拓展三个层次，要求在团队协作下完成从设计、实现到系统验证的完整工程闭环。
- ✓ 统一要求：每组4人，明确模块分工，提交团队协作说明；全程使用HDL实现；须完成功能仿真、硬件调试与系统测试；提交设计报告，进行项目展示与答辩验收。

	题目A	题目B
名称	基于CPM7128的模型计算机系统控制器设计	基于FPGA的处理器设计
平台	TEC-8	Minisys/EGO1/NEXYS4/ TEC-PLUS
架构参照	TEC-8模型机指令集	RISC-V/MIPS/ 自定义ISA
最高层次	流水型硬布线控制器（中断）	流水型CPU+系统集成



题目A：基于Altera CPM7128的模型机系统控制器设计

基础层次—硬布线控制器设计

按照给定指令格式、数据格式与数据通路，以Verilog/VHDL设计实现基于硬布线的顺序模型处理器；支持TEC-8基本指令集（除中断之外）；在TEC-8实验平台完成组装、调试并运行测试程序。

进阶层次—流水型硬布线控制器设计

- ✓ 将顺序控制器改造为流水线结构，分析并解决数据冒险与控制冒险问题；对时钟频率与流水线吞吐量进行量化分析，比较流水前后的性能差异。
- ✓ 实现难点：硬件平台上的冒险处理，需观察、分析并解决吞指令现象。

拓展层次—带中断功能的硬布线控制器设计

- ✓ 在必选题基础上设计实现支持中断响应与返回的硬布线控制器。
- ✓ 设计难点：EPM7128芯片中未接入LIAR和IABUS信号，无法以IAR寄存器实现中断现场保存及返回，需自行设计替代方案。



课程设计任务

题目B：基于FPGA开发板的处理器设计

基础层次—简单CPU设计与实现

设计并实现支持基本指令集的单周期或多周期CPU：以RISC-V（RV32I子集）/MIPS/或自定义ISA为参照，设计指令集、数据通路与控制器；使用Verilog/VHDL实现，在Minisys/EGO1/NEXYS4/TEC-PLUS平台完成综合、布局布线与硬件验证；支持算术、逻辑、访存、跳转等基本指令类型，能够运行简单测试程序。

进阶层次—完整计算机系统设计

在基础CPU基础上集成内存与I/O接口，构建完整可运行系统：完成CPU、内存子系统（含Cache设计或存储层次规划）与基本I/O接口的系统集成；能够支持小型测试程序的完整运行，分析系统性能瓶颈并提出优化方案；引入流水线机制，对时钟频率、CPI与吞吐量进行量化评估。

拓展层次—高性能处理器优化设计

在完整系统基础上进行多维度性能优化与创新探索，从以下方向中选择至少一项深入实现：流水线冒险的完整解决方案（数据前推、分支预测）；Cache替换策略优化与命中率分析；支持浮点运算或乘除法扩展指令；基于RISC-V的自定义ISA扩展设计；面向AI/矩阵计算的自定义加速指令设计（如MAC乘加单元）；进行多方案对比分析，在PPA（功耗·性能·面积）三角约束下给出设计权衡说明。



开源生态——站在巨人肩膀上的处理器设计

从”实现CPU”到”理解AI芯片”：RISC-V：开放指令集架构的崛起

- ✓ 不同于x86/ARM的封闭授权模式，RISC-V指令集完全开源，任何人可设计、实现、商用而无需支付授权费。
- ✓ 国内 “一生一芯” 项目已让数千名本科生完成属于自己的RISC-V处理器流片，部分成果成功完成芯片量产。

开源CPU实现：触手可及的工业级参考

项目	来源	特点
Rocket Chip	UC Berkeley	工业级五级流水线可配置度高
NutShell	中科院计算所	国产开源处理器，文档完善
PicoRV32	独立开发者	轻量级，适合嵌入式场景
一生一芯系列	中科院/各高校	面向本科生教学设计

课程要求： **题目A：** 核心模块须独立完成代码编写、仿真与验证，照搬视为学术不端； **题目B：** 鼓励参考开源实现与AI完成核心模块，但须独立完成需求与接口定义、工程审阅、硬件落地调试与系统联调验收，并能讲清系统设计原理。

学习资源：GitHub搜索：riscv-rocket-chip / NutShell / picorv32；一生一芯官网：oscpu.org

AI辅助工具使用声明表（参考模板）

为落实前后页学术诚信要求，各小组须按下表如实记录AI与开源参考的使用情况，随报告附页一并提交，作为红线规则判定与过程性考核的依据。题目A与题目B的使用边界不同：

题目A（模型机控制器设计）：核心模块须独立完成。AI/开源仅限查阅资料、理解报错、学习语法等辅助环节；核心模块完整代码由AI或开源直接生成后未经独立实现提交，视为学术不端。

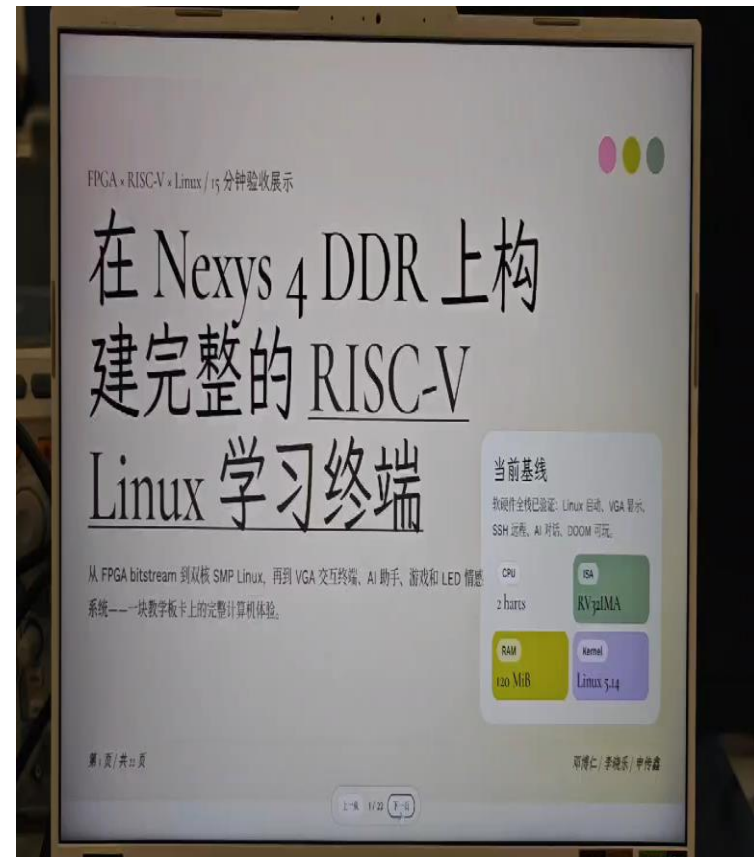
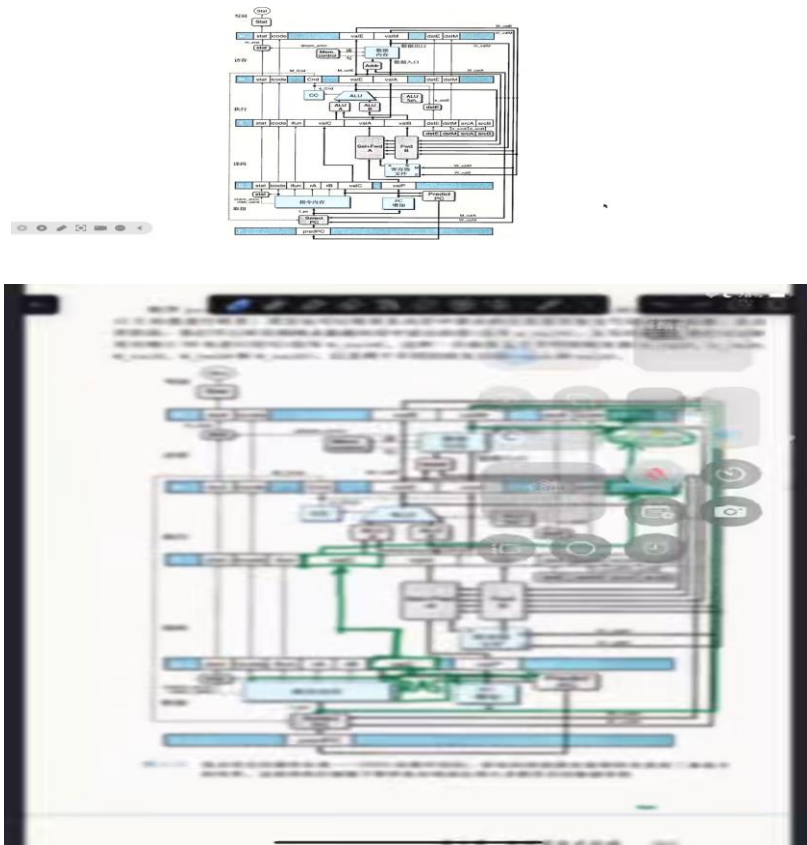
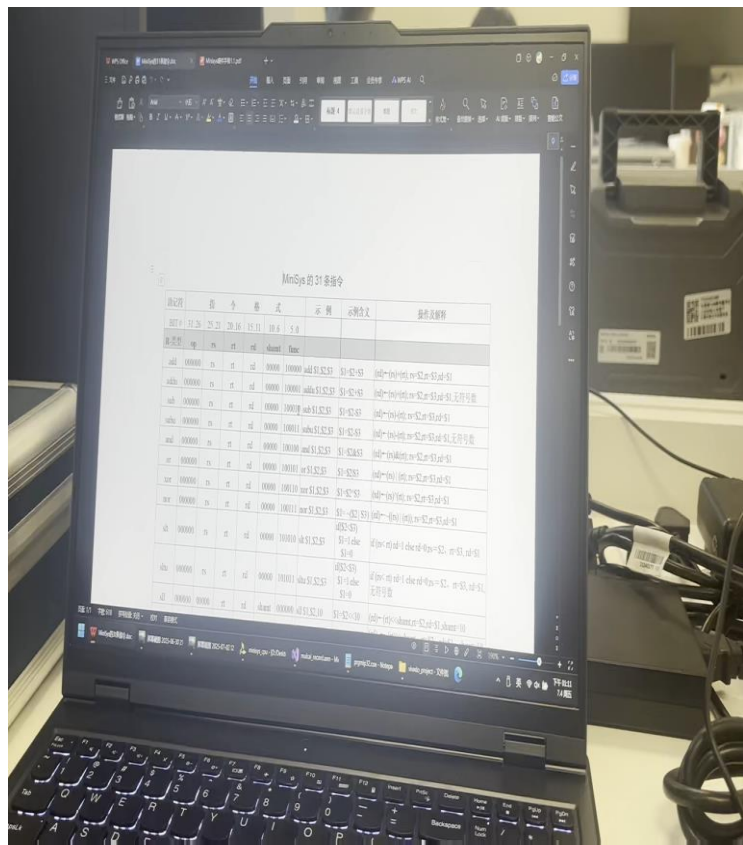
题目B（FPGA处理器设计）：鼓励参考开源实现与AI工具完成核心模块。但须独立完成并如实记录：①需求与指令集/接口定义；②对参考代码的工程审阅与改造说明；③硬件平台落地调试（综合、时序收敛、资源适配）；④系统集成与联调验收。不做审阅整合、无法独立讲解系统设计原理的直接照搬，同样视为学术不端。

使用环节	工具名称	用途说明	生成内容占比估计	人工验证与修改说明
查阅报错信息	ChatGPT/Claude/DeepSeek 等	定位Verilog编译报错原因	0%（仅咨询，未采纳代码）	自行修改代码并复核仿真结果
调试思路建议	（示例，按实际填写）	流水线冒险处理思路参考	约10%（思路提示）	自行设计并编写解决方案，独立完成仿真验证

（按实际使用环节填写）



题目B：基于FPGA开发板的处理器设计示例



- ✓ 用minisys平台/NEXYS4 用FPGA芯片设计基于RISC-V/Y86/MIPS的CPU与完整的计算机系统设计
- ✓ 解析执行官方指定指令
- ✓ 可以执行内置的指令序列; 执行通过I/O输入的指令序列或应用程序, 执行过程中的信息则通过I/O口输出到控制台。
- ✓ LiteX: 基于 Python的 SoC 构建框架, 用Python参数化描述完成CPU选型→SoC组装→HDL生成→综合→烧录



CPLD与FPGA

复杂可编程逻辑器件（CPLD）

- 可编程逻辑宏单元与互联矩阵组成的PLD
- 内部逻辑宏单元和互连矩阵组成粗粒度结构，触发器有限而乘积项丰富
- 可以进行程序性的规划、刻录
- 适用于中等规模的数字电路，具有速度快、时序确定、保密性好等优点，但有功耗高、集成度低、资源利用率低等缺点。

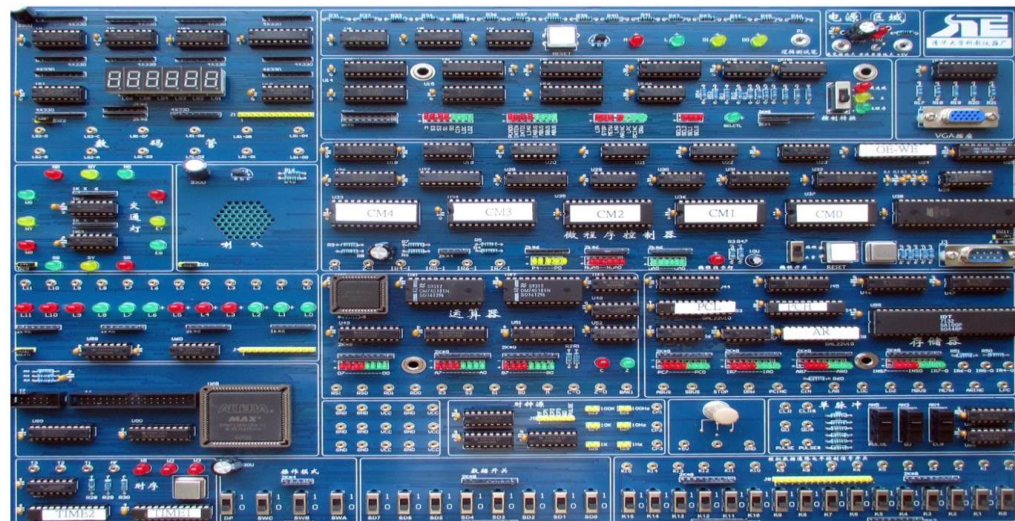
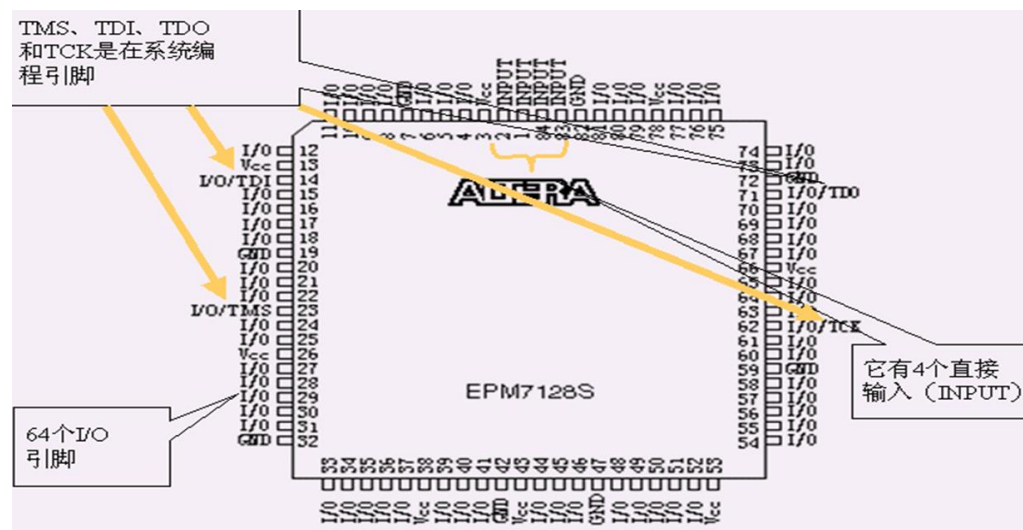
现场可编程逻辑阵列（FPGA）：

- 大量的可编程逻辑单元和可编程互连结构组成的PLD
- 内部逻辑功能块和互联组成细粒度结构，触发器丰富
- 可以通过软件进行配置和修改
- 适用于大规模的数字电路，具有灵活性高、资源利用率高、可重复使用等优点，但有速度慢、时序不可测、保密性差等缺点。



可选平台

- **TEC-8**计算机硬件综合实验系统，搭载ALTERA EPM7128,开发平台quartus。
- **EPM7128SLC84-15**是一种复杂可编程逻辑器件（CPLD），属于Altera公司的MAX 7000S系列。它的封装为PLCC，芯片制程是0.5微米，采用CMOS工艺。CMOS门电路由P型与N型MOS管构成。它的晶体管数量是2500个，可以实现128个逻辑宏单元和68个输入/输出端。

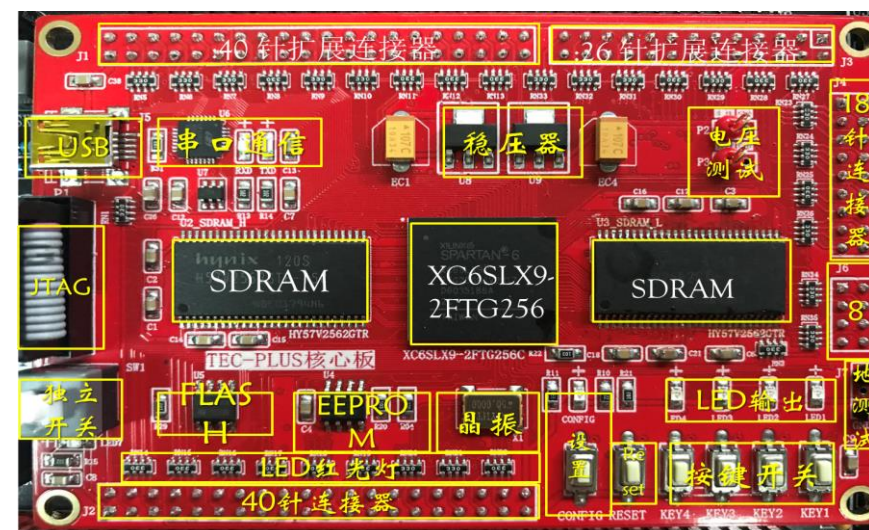
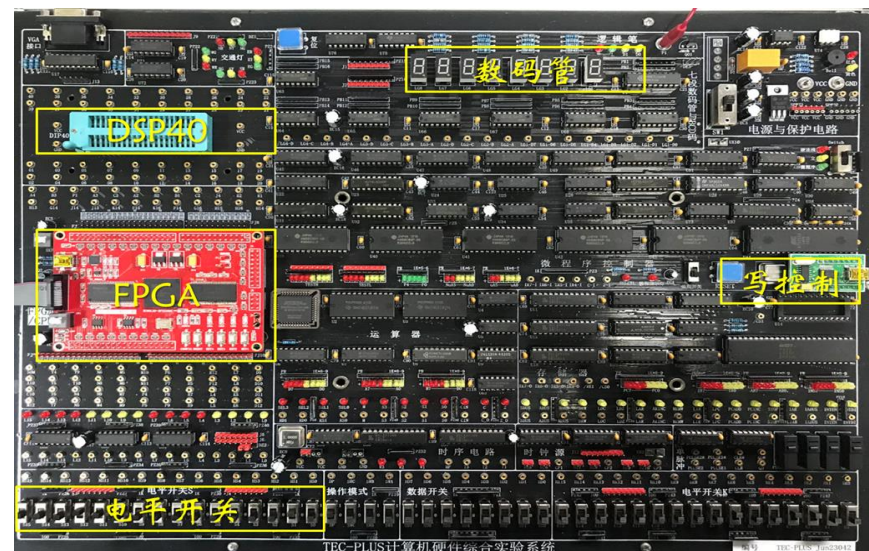




可选平台

TECPLUS计算机硬件综合实验系统，开发平台ISE14.7
XILINX Spartan-6 XC6SLX9-2FTG256 是一款可编程逻辑
FPGA。

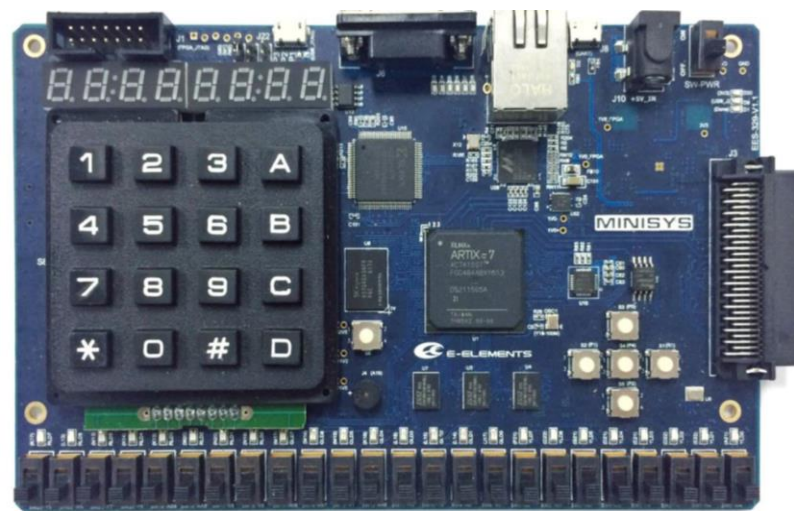
1. 芯片制程：采用45纳米（0.045微米）的低功耗铜工艺。
2. 芯片规模： 逻辑单元数：9,152个，内存容量：576 Kb，
DSP切片数：16个，最大I/O数200个。





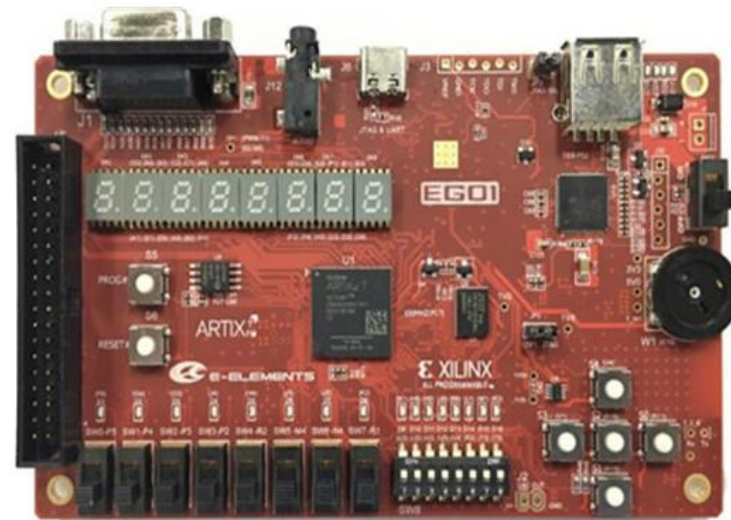
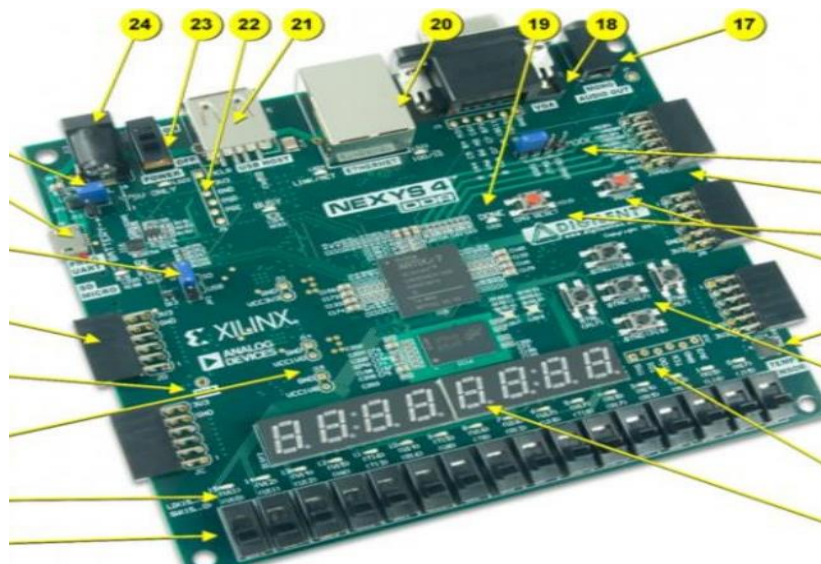
Minisys实验板是一个以**Xilinx Artix-7TM**系列FPGA（XC7A100TFGG484C-1）为主芯片实验平台。开发平台**VIVADO**。

- 器件系列：Artix-7
- 逻辑单元/配置逻辑块数：101,440
- 总RAM位数：4,976,640
- I/O引脚数：285
- 应用领域：适用于各类成本和功耗敏感型应用，包括软件定义无线电、机器视觉照相以及低端无线回传等。





可选平台



Xilinx EGO1和**NEXYS 4**是搭载Artix-7系列FPGA芯片的平台，提供了丰富的拨码开关与LED，外设功能部份有音频、蓝牙、XADC/ DAC、SRAM等功能。

开发平台**VIVADO**



设计思路与技术要点

硬件描述语言（HDL）选择：VHDL或Verilog HDL

VHDL(DARPA美国国防高级计划研究局设计)

优点：强类型系统，数据类型严格减少错误；模块化设计，可读性高，代码结构清晰，便于维护和理解。

缺点：语法复杂；应用领域市场占有率低。

适合设计需要强类型系统和多种仿真方法的开发。

Verilog HDL(Gateway Design Automation公司设计)

优点：语法简洁；市场占有率高；灵活性高，适合硬件实现和电路的合成与优化。

缺点：弱类型系统，容易出现类型错误；模块化支持较弱；语法简洁，代码的可读性和维护性低。

适合更侧重于数字电路的行为模拟和硬件实现的开发。



设计思路与技术要点

工程类项目开发参考

- ✓ **需求分析：**明确项目的功能需求、性能指标、用户需求和约束条件。
- ✓ **系统设计重点：**制定系统的总体架构和详细设计方案。架构设计：确定系统的模块划分、接口定义和数据流。详细设计：为每个模块编写详细的设计文档，包括算法、数据结构和接口。技术实现重点：根据设计文档进行编码和实现，确保代码的可读性和可维护性。
- ✓ **测试与验证重点：**确保系统的功能和性能符合需求。做好单元测试、集成测试，系统测试，验证其性能和稳定性。



设计思路与技术要点

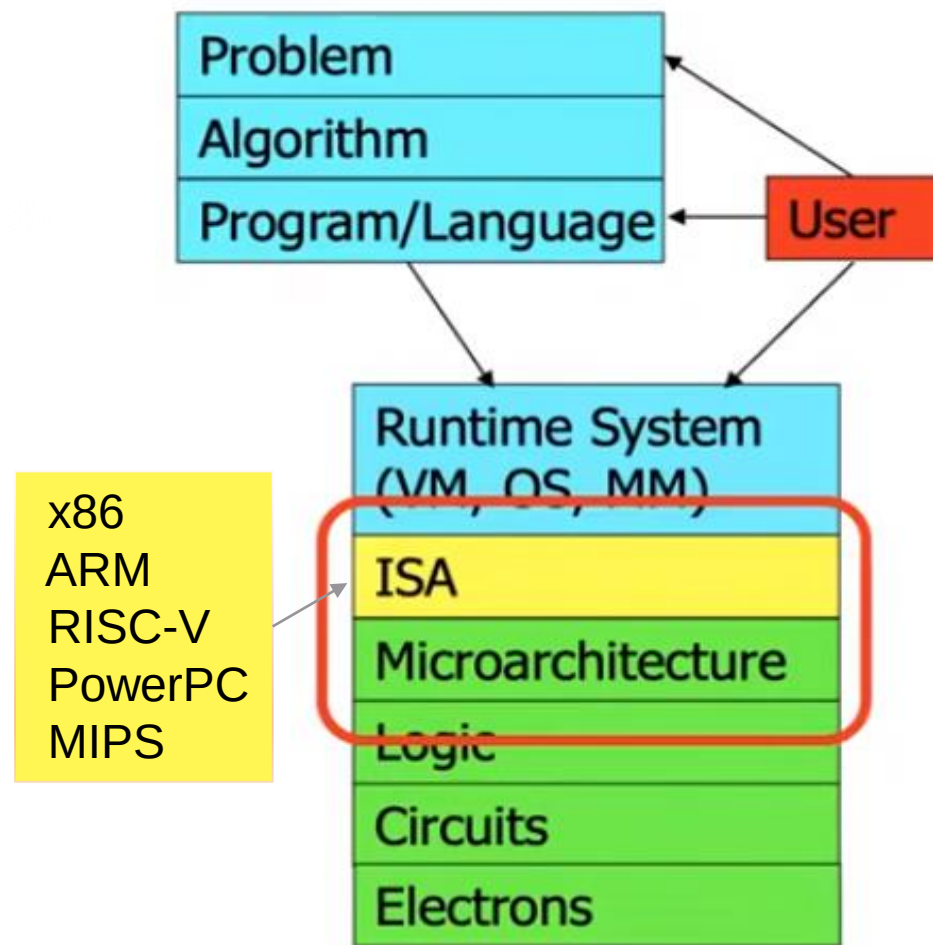
AI辅助EDA——行业趋势简介

- ✓ **趋势一：AI辅助代码生成与调试** 主流EDA厂商（Cadence、Synopsys）与开源社区已推出AI辅助RTL代码生成、报错定位、时序违例分析等功能，大幅降低了HDL编码门槛。
- ✓ **趋势二：自然语言转RTL** 学术界与工业界正在探索"自然语言描述→HDL代码"的自动生成工具，但目前仍需工程师验证时序、资源与功能正确性，无法完全替代底层电路理解。
- ✓ **趋势三：AI辅助验证与优化** 在功能仿真、覆盖率分析、布局布线参数调优等环节，AI正在成为提升效率的辅助工具，但设计决策（架构选择、性能权衡）仍依赖工程师的判断力。
- ✓ **课程启示：** 本课程要求独立完成核心模块设计，正是为了让你在AI大幅降低"写代码"门槛的时代，仍然具备"看懂电路、独立调试、承担设计责任"的核心能力——这是AI辅助工具目前还无法替代的部分。



知识衔接与重点解析

- ✓ 整个体系结构的一切均和**权衡**相关。
- ✓ 计算机软硬件的区分：程序员用户可见（可直接操作）：主存储器、程序指针、寄存器；程序员不可见的：为体系结构状态部分，硬件契约。
- ✓ 指令集架构（ISA）与微架构（Microarchitecture）
- ✓ 冯诺依曼结构两大特征：存储指令、顺序指令处理
- ✓ 指令处理引擎由两部分构成：数据路径data path 和控制逻辑control logic



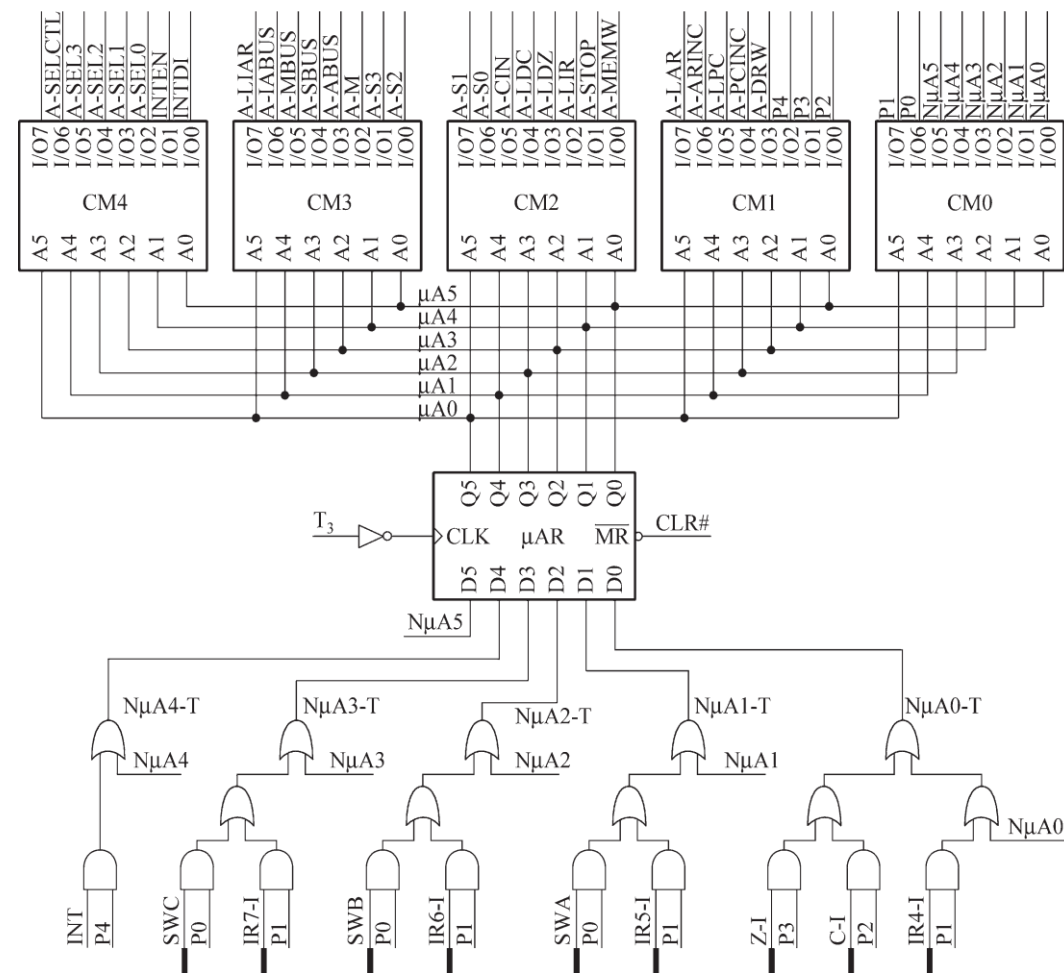


知识衔接与重点解析

名称	助记符	功能	指令格式					
			IR7	IR6	IR5	IR4	IR3 IR2	IR1 IR0
加法	ADD Rd, Rs	$Rd \leftarrow Rd + Rs$	0001				Rd	Rs
减法	SUB Rd, Rs	$Rd \leftarrow Rd - Rs$	0010				Rd	Rs
逻辑与	AND Rd, Rs	$Rd \leftarrow Rd \text{ and } Rs$	0011				Rd	Rs
加 1	INC Rd	$Rd \leftarrow Rd + 1$	0100				Rd	XX
取数	LD Rd, [Rs]	$Rd \leftarrow [Rs]$	0101				Rd	Rs
存数	ST Rs, [Rd]	$Rs \rightarrow [Rd]$	0110				Rd	Rs
C 条件转移	JC addr	如果 C=1, 则 $PC \leftarrow @ + \text{offset}$	0111				offset	
Z 条件转移	JZ addr	如果 Z=1, 则 $PC \leftarrow @ + \text{offset}$	1000				offset	
无条件转移	JMP [Rd]	$PC \leftarrow Rd$	1001				Rd	XX
输出	OUT Rs	$DBUS \leftarrow Rs$	1010				XX	Rs
中断返回	IRET	返回断点	1011				XX	XX
关中断	DI	禁止中断	1100				XX	XX
开中断	EI	允许中断	1101				XX	XX
停机	STP	暂停运行	1110				XX	XX

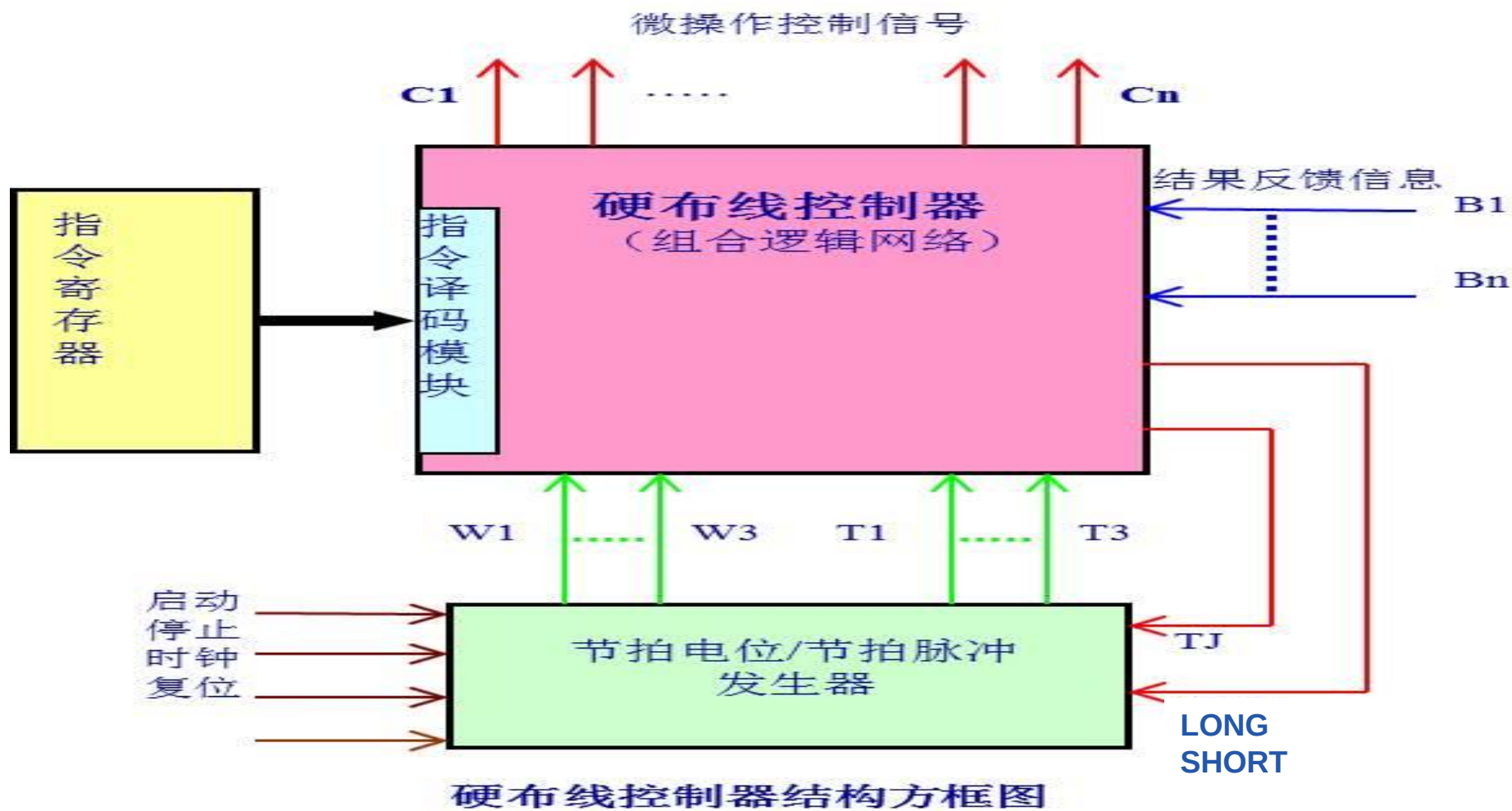


TEC-8 (TEC-PLUS)模型计算机





知识衔接与重点解析

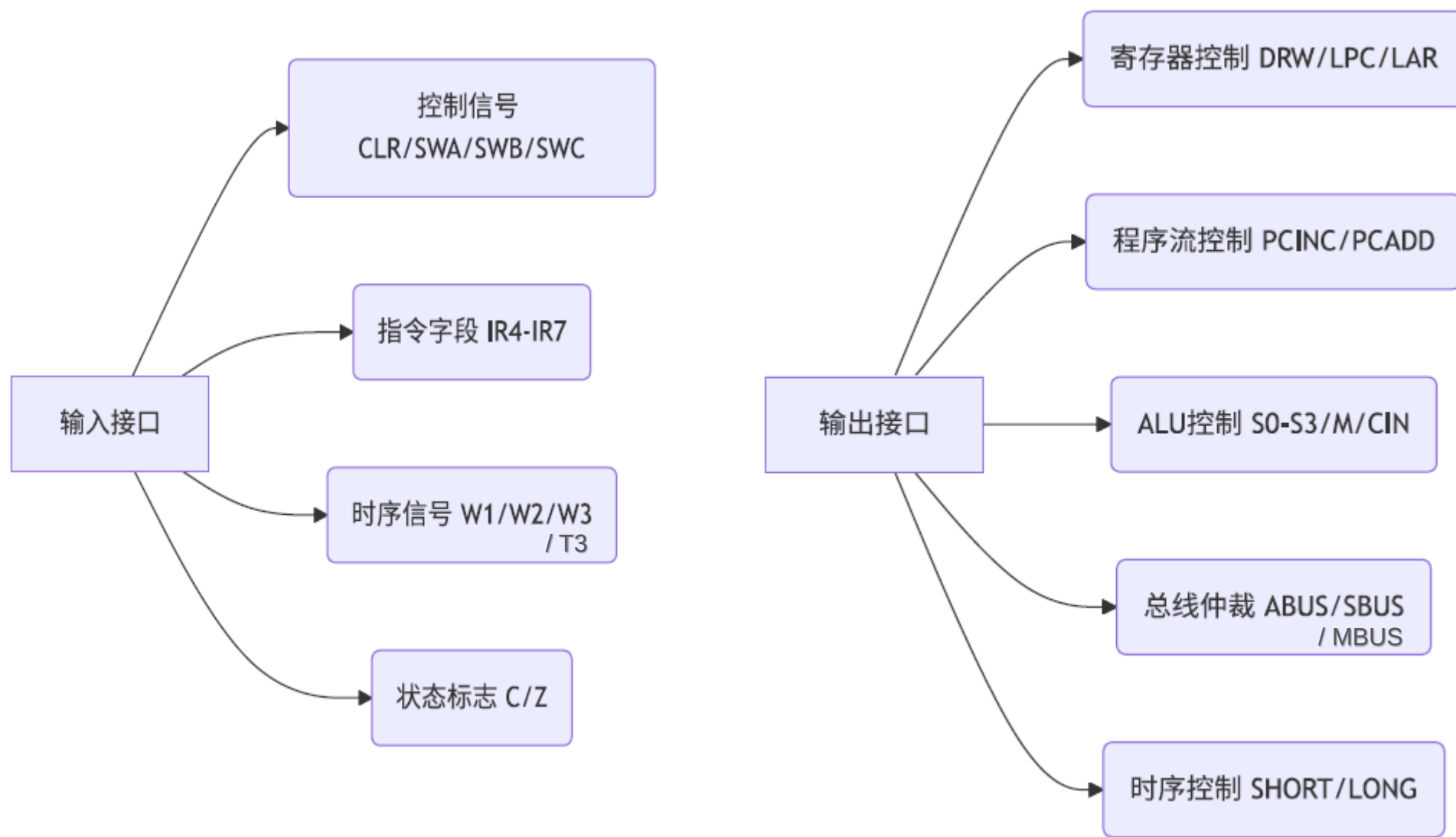


$$C = f(E_n, I_m, B_j, W_i, T_k)$$

控制器结构图

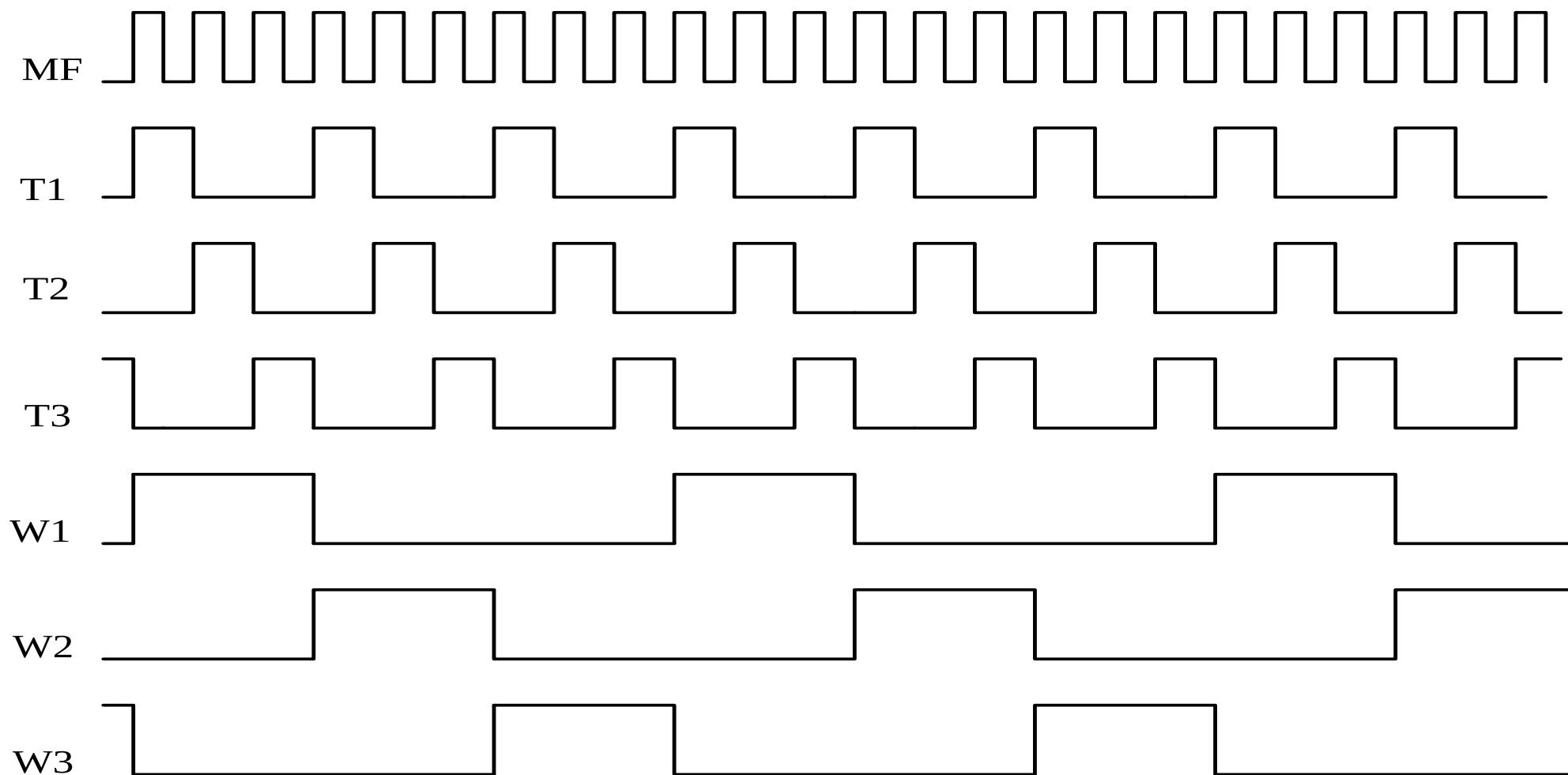


TEC-8(PLUS)控制器输入输出接口规范





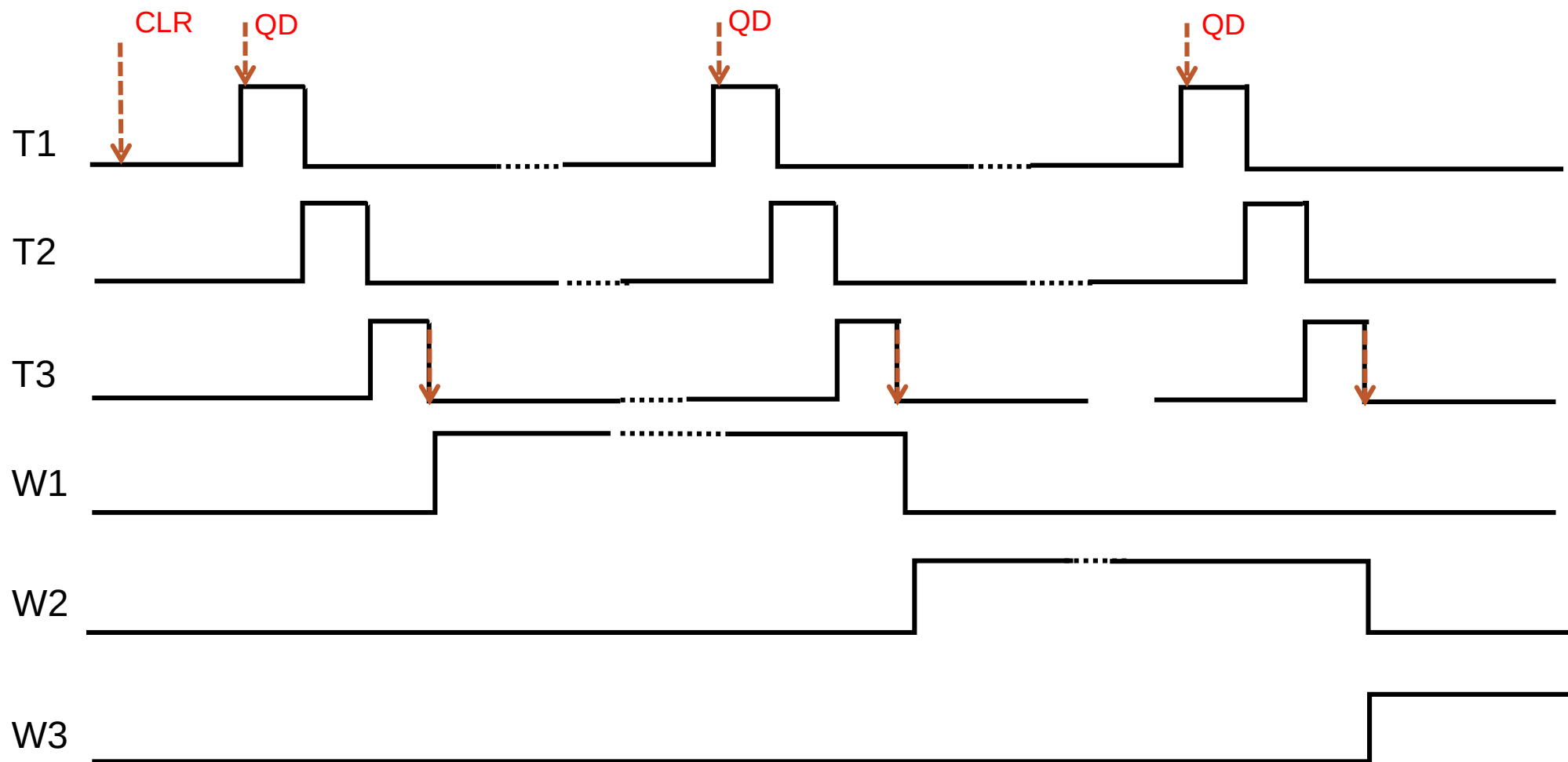
基本时序波形（连续）



MF周期 $1\mu\text{s}$ ，占空比50%，T1~T3脉宽 $1\mu\text{s}$ 。微指令周期 $3\mu\text{s}$ 。



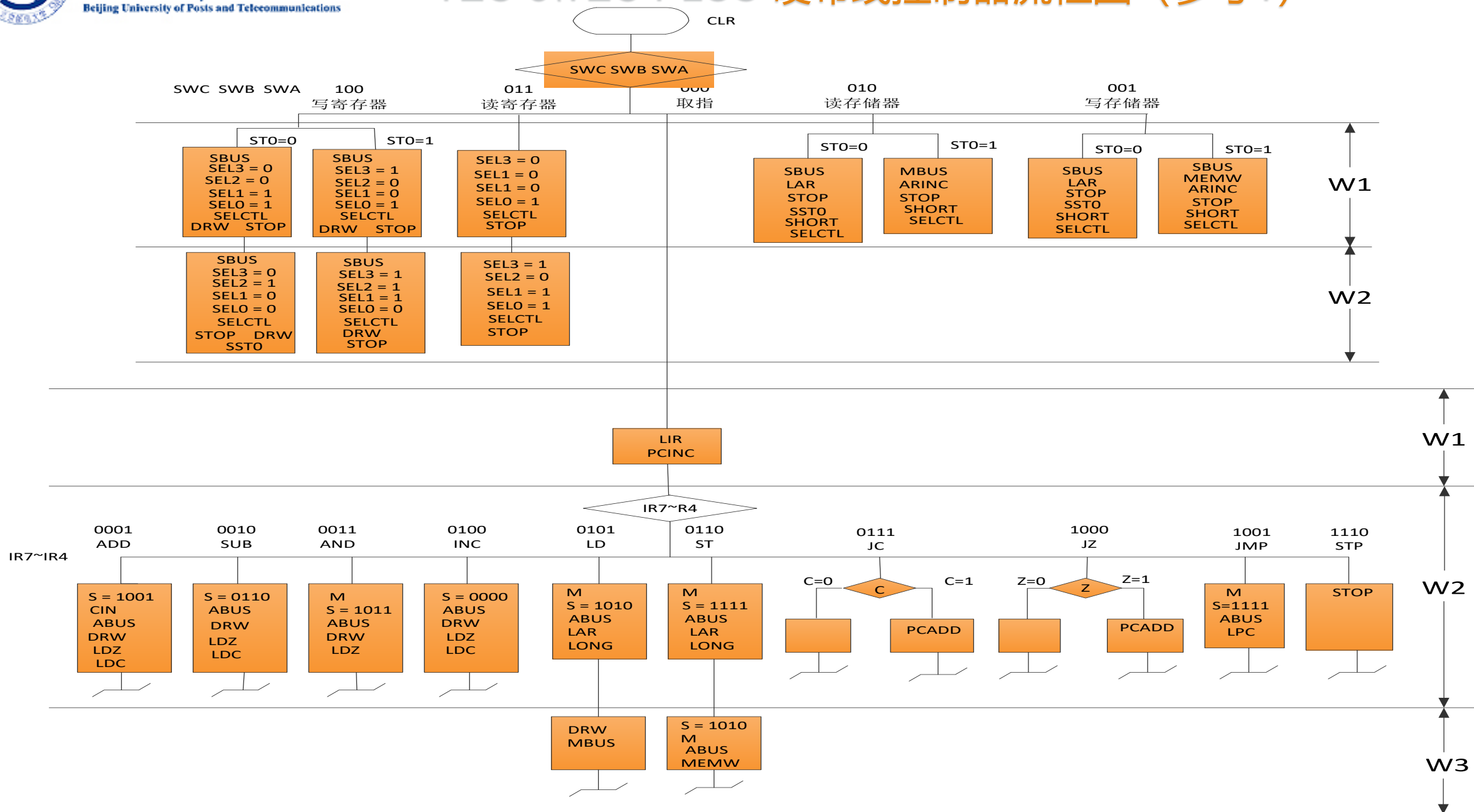
基本时序波形（单拍）



MF周期 $1\mu\text{s}$ ，占空比50%，T1~T3脉宽 $1\mu\text{s}$ 。微指令周期 $3\mu\text{s}$ 。

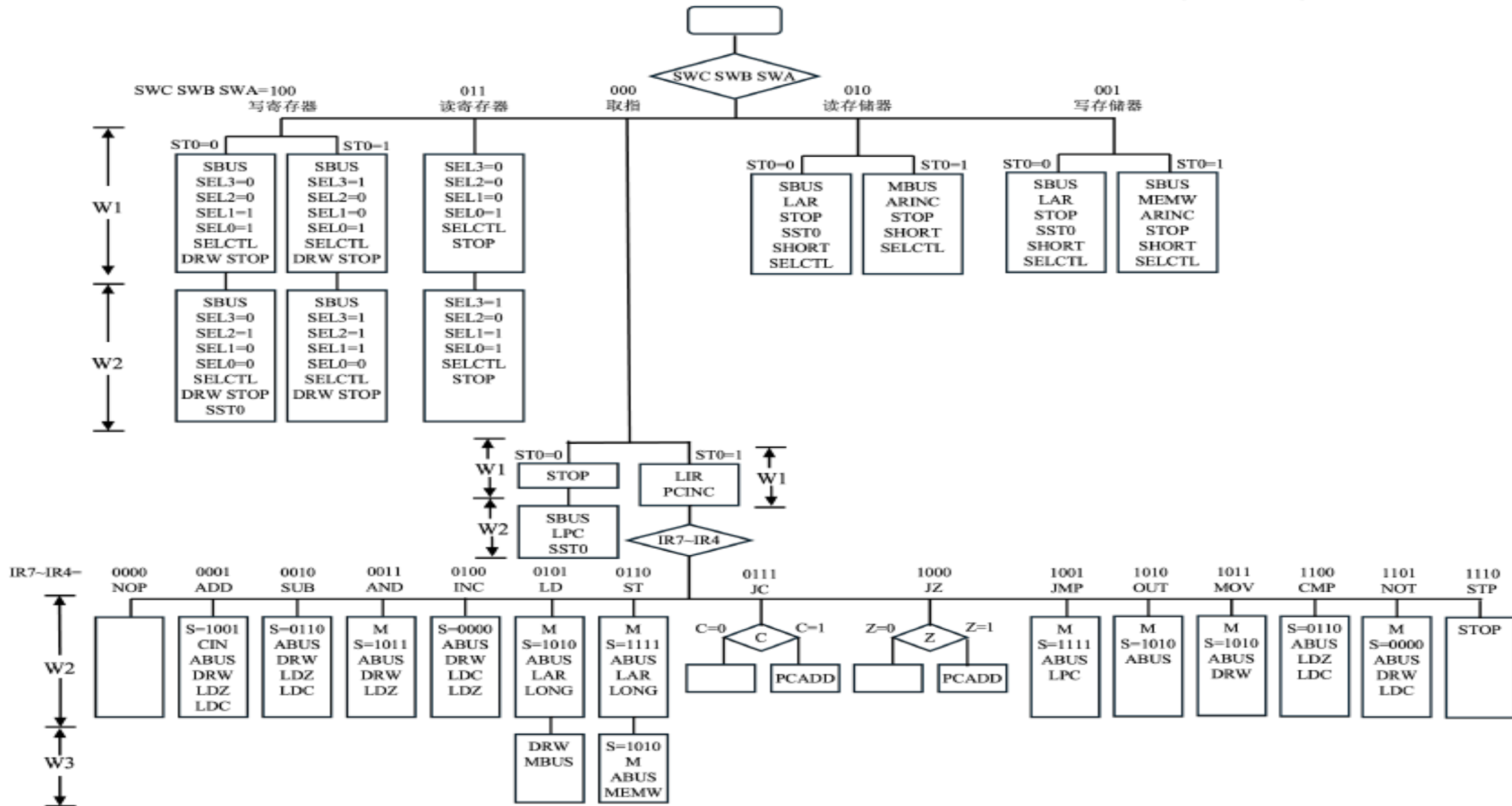


TEC-8\TEC-PLUS 硬布线控制器流程图 (参考1)





TEC-8\TEC-PLUS 硬布线控制器流程图 (参考2)

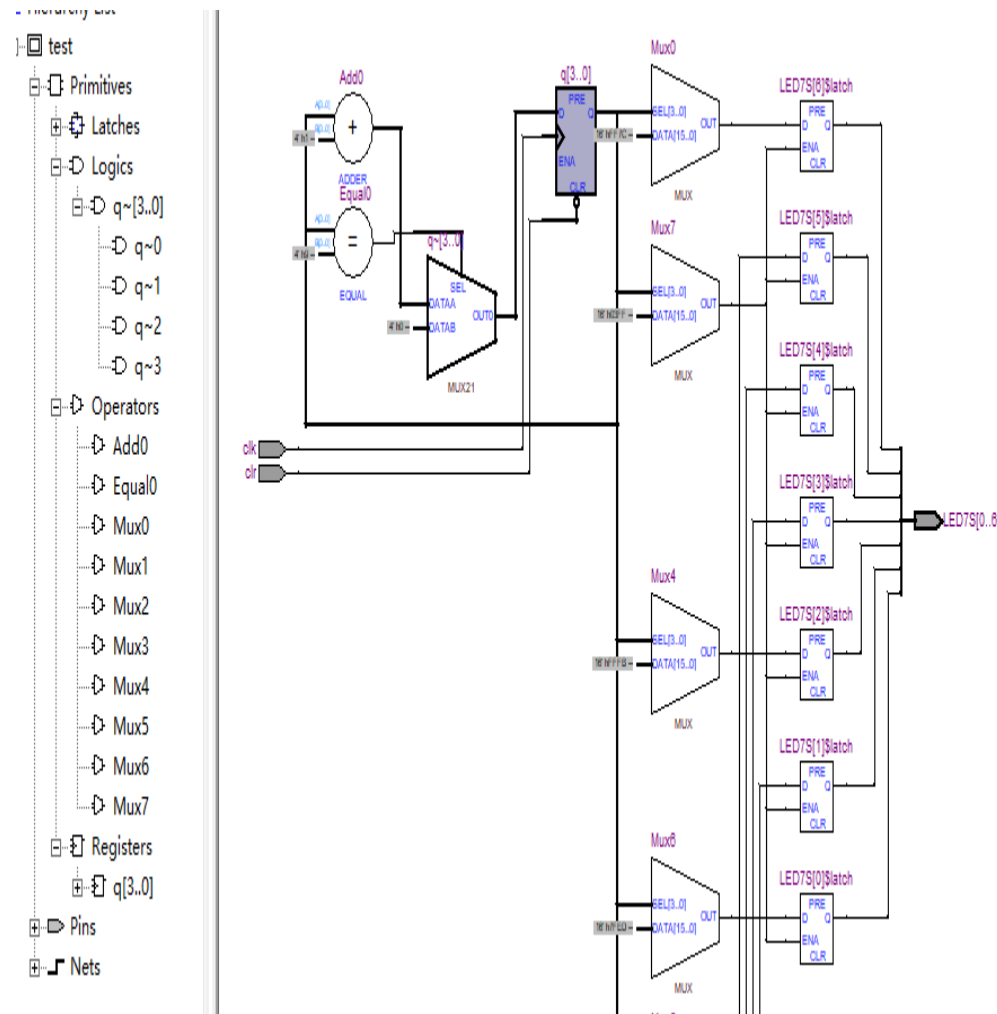




设计思路与技术要点

硬件描述语特点与调试技巧

- ✓ **并行性与硬件特性**: 硬件描述语言的并行性, 非特殊语句全并行; 设计时要有电路概念。
- ✓ **Case语句与IF语句**: 两种条件分支语句, 尽量多用case语句。
- ✓ **多描述确定硬件电路, 慎用有限状态机**: 有限状态机为综合工具实现, 状态转移复杂不明时易发生不可知错误。
- ✓ **通过 Tools/Netlist Viewers/RTL viewer 查看布线**, 优化设计以提高综合布线效率。
- ✓ **硬件描述语言设计性能权衡**: 面积 (资源) 与速度的权衡, 寄存器数量与组合逻辑运算单元数量的考量。





1. 代码规范与可读性

- ✓ 使用清晰的命名规则：模块、信号、变量要有意义。
- ✓ 使用注释：特别是状态机、关键信号和复杂逻辑，增加代码可读性。
- ✓ 避免使用 assign 连续赋值进行大规模组合逻辑，推荐 always @(*) 来描述组合逻辑。

2. 严格使用可综合语句

- ✓ 仅使用 “always/`assign/if/case” 等可综合语句；避免 “initial/fork-join/time/force-release” 等仿真专用语句；禁止在RTL代码中使用 `# 延时（仅用于测试激励）；不要使用循环语句。

reg [7:0] data = 8'hFF; // initial 语句在综合中无效

```
always @(posedge clk or negedge reset_n) begin
    if (!reset_n)
        data <= 8'hFF; // 通过复位进行初始化
end
```




3. 时序设计

- ✓ 所有寄存器时钟同步;
- ✓ 仅使用一个全局时钟 (如 clk) 驱动寄存器, 避免异步时钟域问题;
- ✓ 复位策略尽量使用异步复位同步释放;
- ✓ 采用非阻塞赋值 ($\leq$) 更新寄存器;

```
always @(posedge clk or negedge
reset_n) begin
    if (!reset_n)
        count <= 0;
    else
        count <= count + 1;
end
```

4. 组合逻辑设计

- ✓ 确保所有组合逻辑都有完整的赋值, 防止综合工具产生锁存器latch

```
always @(*) begin
    case (sel)
        2'b00: out = in0;
        2'b01: out = in1;
        2'b10: out = in2;
        2'b11: out = in3;
        default: out = 0; // 避免 latch
    endcase
end
```



5.阻塞与非阻塞

- ✓阻塞赋值：语句按顺序执行，适用于组合逻辑建模。
- ✓非阻塞赋值：语句并行执行，适用于时序逻辑建模。
- ✓避免在同一个 always 块中混用阻塞和非阻塞赋值，以防止产生不可预期的行为。

6.建模选择

- ✓初学推荐行为级建模（模块+过程块描述）；熟练可使用数据流建模（连续赋值语句assign+运算符）；慎用系统级建模（层级模块调用）



7.合理使用有限状态机 (FSM)

✓使用 case 语句避免复杂 if-else 结构。

```
parameter IDLE = 2'b00, READ = 2'b01, WRITE = 2'b10;  
reg [1:0] state;  
always @(posedge clk or negedge reset_n) begin  
    if (!reset_n)  
        state <= IDLE;  
    else begin  
        case (state)  
            IDLE: if (start) state <= READ;  
            READ: if (done) state <= WRITE;  
            WRITE: if (finish) state <= IDLE;  
            default: state <= IDLE;  
        endcase  
    end  
end
```

8.信号类型与操作符

✓建议仅用reg、wire型

✓数制尽量采用二进制

✓不做除法与取模运算



9. 常见错误规避

错误类型	解决方案
锁存器生成	确保组合逻辑完全条件判断
多时钟域冲突	严格同步处理跨时钟域信号
仿真-综合不一致	避免在RTL中使用不可综合结构
不定态传播	设置默认复位值，避免X态传播

10. 优化策略（资源与速度的权衡）

- ✓ 关键路径速度优化：
 - 复杂组合逻辑拆分为流水线结构
 - 优先级编码改用case语句
- ✓ 面积（资源）优化：
 - 资源共享（时分复用）
 - 状态机合并冗余状态

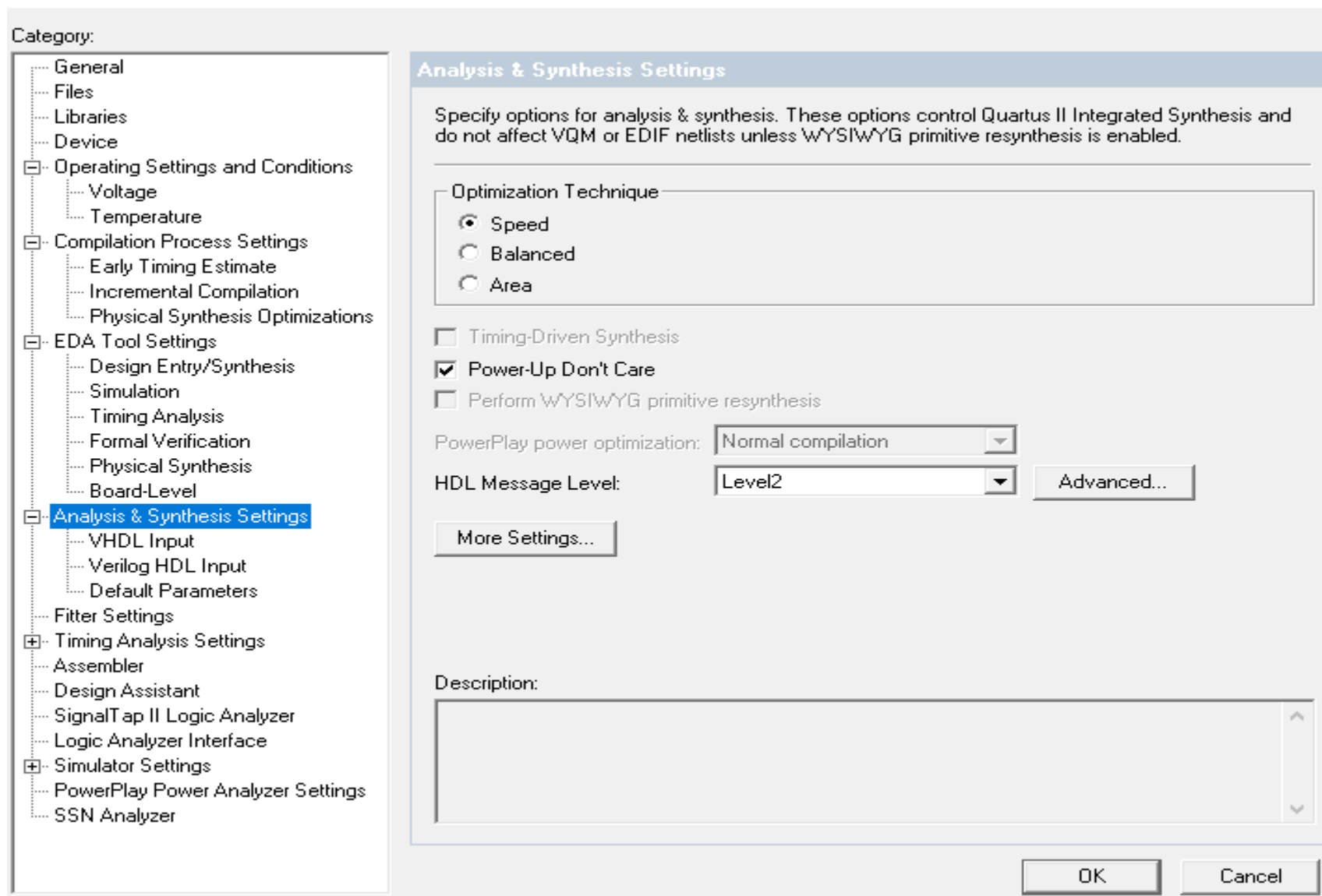


11.quartus优化选项

Speed(速度)

Balance (平衡)

Area (资源)





硬件描述语言典型不当设计举例

✓硬件描述语言的
**并行性，非特殊语
句全并行**

✓**不可初始化。完
善复位置位，与分
支描述**

```
if (!CLR)
    begin
        {ARINC, CIN, DRW, LPC, LAR, LIR, LDZ, LDC, PCINC, PCADD,
        SELCTL, M, MEMW, STOP, SHORT, LONG, ABUS, SBUS, MBUS,
        SST0} = 0;
        {S,SEL}=0;
    end
else
    begin
        {ARINC, CIN, DRW, LPC, LAR, LIR, LDZ, LDC, PCINC, PCADD,
        SELCTL, M, MEMW, STOP, SHORT, LONG, ABUS, SBUS, MBUS,
        SST0} = 0;
        S,SEL}=0;
    case (SWC_SWB_SWA)
        .....
```



TEC-8平台
EPM7128
硬布线控制器部分引脚

信号	方向	引脚号	信号	方向	引脚号
CLR#	输入	1	MEMW	输出	27
T3	输入	83	STOP	输出	28
SWA	输入	4	LIR	输出	29
SWB	输入	5	LDZ	输出	30
SWC	输入	6	LDC	输出	31
IR4	输入	8	CIN	输出	33
IR5	输入	9	S0	输出	34
IR6	输入	10	S1	输出	35
IR7	输入	11	S2	输出	36
W1	输入	12	S3	输出	37
W2	输入	15	M	输出	39
W3	输入	16	ABUS	输出	40
C	输入	2	SBUS	输出	41
Z	输入	84	MBUS	输出	44
DRW	输出	20	SHORT	输出	45
PCINC	输出	21	LONG	输出	46
LPC	输出	22	SEL0	输出	48
LAR	输出	25	SEL1	输出	49
PCADD	输出	18	SEL2	输出	50
ARINC	输出	24	SEL3	输出	51
SELCTL	输出	52			



附录

信号	方向	引脚号	信号	方向	引脚号
CLR#	输入	F6	C	输入	B5
T ₃	输入	C10	Z	输入	A5
SWA	输入	K11	DRW	输出	A13
SWB	输入	F7	PCINC	输出	T14
SWC	输入	J12	LPC	输出	C5
IR ₄	输入	G12	LAR	输出	E6
IR ₅	输入	E11	PCADD	输出	C11
IR ₆	输入	J11	ARINC	输出	P15
IR ₇	输入	E12	SELCTL	输出	B10
W ₁	输入	F9	MEMW	输出	C8
W ₂	输入	K12	STOP	输出	C9
W ₃	输入	N5	LIR	输出	D8
LDZ	输出	R16	SBUS	输出	A8
LDC	输出	E3	MBUS	输出	A14
CIN	输出	R15	SHORT	输出	A6
S ₀	输出	R14	LONG	输出	B6
S ₁	输出	H11	SELO	输出	C6
S ₂	输出	T15	SEL1	输出	A11
S ₃	输出	C13	SEL2	输出	C7
M	输出	A7	SEL3	输出	A12
ABUS	输出	B8			

TEC-PLUS硬连线控制器部分引脚定义



参考资料：

- 《计算机组成原理(第七版)》--戴志涛 白中英等，科学出版社
- 《数字逻辑与数字系统解题库实验指导》--白中英 杨春武 主编，科学出版社
- 《TEC-8计算机组成和数字逻辑实验系统实验指导书》--清华大学科教仪器厂
- 《TEC-PLUS计算机组成原理实验指导书》--清华大学科教仪器厂
- 课程QQ群：各平台相关资料见群内分享
- AI辅助学习资源使用建议：可借助AI工具（如ChatGPT、Claude、DeepSeek等）辅助理解Verilog/VHDL语法、定位编译报错原因、学习优秀开源实现的设计思路；但核心模块（运算器、控制器、流水线寄存器等）的设计与调试须独立完成，并按第13页《AI辅助工具使用声明表》如实记录使用过程。



北京邮电大学

Beijing University of Posts and Telecommunications

谢谢！



厚德博学 敬业乐群